

Lab 1 - Query plans a indexovanie v PostgreSQL

Skripty su ulozene v `1/sql/` a vystupy v `1/outputs/`. Kazdy experiment ma vlastny SQL subor a vlastny vystup, aby sa dali merania zopakovat bez prepisovania inych experimentov.

00 - Inspect tabulky `documents`

Skript: `sql/00_inspect.sql`

Vystup: `outputs/00_inspect.txt`

Zistenia:

- Tabulka `documents` ma 351224 riadkov.
- Velkost tabulky je 102 MB.
- Na tabulke neboli ziadne indexy.

Tento stav je dolezity ako baseline pred meranim dotazov. Ak tabulka nema index, PostgreSQL nema inu lacnu cestu k riadkom podla hodnoty v stlpci `supplier` a musi citat celu tabulku.

01 - Filter `supplier = 'SPP'` bez indexu

Skript: `sql/01_supplier_no_index.sql`

Vystup: `outputs/01_supplier_no_index.txt`

Dotaz:

```
EXPLAIN ANALYZE
SELECT *
FROM documents
WHERE supplier = 'SPP';
```

Ocakavany plan bez indexu je `Seq Scan on documents`. Databaza cita riadky tabulky postupne a na kazdom riadku vyhodnoti podmienku `supplier = 'SPP'`.

Uz namerany prvy beh:

- Plan: Seq Scan on documents
- Vratene riadky: 32
- Rows Removed by Filter : 351192
- Total runtime : 48.276 ms
- psql Time : 52.379 ms

Nove meranie v outputs/01_supplier_no_index.txt :

Beh	Plan	Total runtime	psql Time
1	Seq Scan	48.250 ms	48.836 ms
2	Seq Scan	50.370 ms	50.680 ms
3	Seq Scan	49.336 ms	49.661 ms

Median Total runtime : 49.336 ms.

Interpretacia: dotaz je velmi selektivny, pretoze z 351224 riadkov vracia iba 32. Bez indexu je vsak PostgreSQL nuteny prejsť celu tabuľku. Riadok Rows Removed by Filter: 351192 ukazuje, ze vacsina prace bola citanie a filtrovanie riadkov, ktore sa nakoniec nepouzili.

02 - Filter supplier = 'SPP' s indexom

Skript: sql/02_supplier_with_index.sql

Vystup: outputs/02_supplier_with_index.txt

Index:

```
CREATE INDEX index_documents_on_supplier ON documents(supplier);
```

Namerany plan po vytvorení indexu je Bitmap Index Scan + Bitmap Heap Scan . Index umožní databáze najst položky s hodnotou SPP ; bitmap heap scan potom docita prislusne riadky z tabuľky.

Meranie v outputs/02_supplier_with_index.txt :

Beh	Plan	Total runtime	psql Time
1	Bitmap Index Scan + Bitmap Heap Scan	0.050 ms	0.455 ms
2	Bitmap Index Scan + Bitmap Heap Scan	0.032 ms	0.292 ms

Beh	Plan	Total runtime	psql Time
3	Bitmap Index Scan + Bitmap Heap Scan	0.027 ms	0.212 ms

Median Total runtime : 0.032 ms.

Porovnanie: podľa medianu Total runtime bol dotaz po pridani indexu priblizne 1542x rychlejsi. Podľa psql Time , kde je vidiet aj klientsky overhead, bol priblizne 170x rychlejsi. Dolezite je, ze databaza uz necitala celu 102 MB tabulku.

03 - Range filter nad total_amount bez indexu

Skript: sql/03_total_amount_no_index.sql

Vystup: outputs/03_total_amount_no_index.txt

Dotaz:

```
EXPLAIN ANALYZE
SELECT *
FROM documents
WHERE total_amount > 100000
AND total_amount <= 999999999;
```

Pred meranim skript odstrani indexy index_documents_on_supplier a index_documents_on_total_amount , aby bol stav tabulky cisty pre baseline bez indexu.

Meranie v outputs/03_total_amount_no_index.txt :

Beh	Plan	Total runtime	psql Time
1	Seq Scan	53.322 ms	53.931 ms
2	Seq Scan	53.652 ms	54.012 ms
3	Seq Scan	52.948 ms	53.338 ms

Median Total runtime : 53.322 ms. Dotaz vrátil 15707 riadkov a odfiltroval 335517 riadkov.

04 - Range filter nad total_amount s indexom

Skript: sql/04_total_amount_with_index.sql

Vystup: outputs/04_total_amount_with_index.txt

Index:

```
CREATE INDEX index_documents_on_total_amount ON documents(total_amount);
```

Meranie v outputs/04_total_amount_with_index.txt :

Beh	Plan	Total runtime	psql Time
1	Bitmap Index Scan + Bitmap Heap Scan	7.036 ms	7.592 ms
2	Bitmap Index Scan + Bitmap Heap Scan	6.217 ms	6.628 ms
3	Bitmap Index Scan + Bitmap Heap Scan	6.437 ms	6.803 ms

Median Total runtime : 6.437 ms.

Interpretacia: index pomohol, ale menej dramaticky ako pri `supplier = 'SPP'`. Range dotaz vrátil 15707 riadkov, preto databaza musela okrem citania indexu docitat relativne vela riadkov z heapu. Planner zvolil bitmap plan, pretoze je vhodny, ked index najde viac riadkov: najprv sa posklada bitmapa pozicii riadkov z indexu a potom sa riadky citaju z tabulky efektivnejsie.

Porovnanie: podla medianu Total runtime bol range dotaz s indexom priblizne 8.3x rychlejsi.

05 az 08 - INSERT benchmark s 0, 1, 2 a 3 indexmi

Skripty:

- sql/05_insert_no_index.sql
- sql/06_insert_one_index.sql
- sql/07_insert_two_indexes.sql
- sql/08_insert_three_indexes.sql

Kazdy skript vkladá batch 10000 riadkov pomocou jednoducheho vzoru:

```
BEGIN;  
EXPLAIN ANALYZE  
INSERT INTO documents  
SELECT *  
FROM documents  
LIMIT 10000;  
ROLLBACK;
```

ROLLBACK znamená, že INSERT sa realne vykona a zmeria, vrátane práce nad indexmi, ale data po experimente v tabulke nezostanu. Použitie `SELECT *` je v tomto labovom image v poriadku, lebo tabulka nema primary key index na `id` a zmeny sa aj tak rollbacknu. Na cvicenie staci jeden beh v kazdom subore.

Meranie:

Experiment	Indexy	Total runtime
05	ziadne	31.333 ms
06	supplier	152.790 ms
07	supplier , total_amount	234.801 ms
08	supplier , total_amount , published_on	254.727 ms

Interpretacia: INSERT bez indexov bol najrychlejsi. Po pridani indexov sa INSERT spomalil, lebo PostgreSQL pri kazdom vlozenom riadku nevkлада data iba do tabulky, ale musi aktualizovat aj kazdy index.

09 a 10 - INSERT benchmark: nizka vs vysoka kardinalita

Skripty:

- `sql/09_insert_low_cardinality_index.sql`
- `sql/10_insert_high_cardinality_index.sql`

Pre nizku kardinalitu je pouzity index na `type`, kde su 4 rozne hodnoty. Pre vysoku kardinalitu je pouzity index na `supplier`, kde je 138789 roznych hodnot. Cielom je zistit, ci samotna kardinalita indexovaného stlpca viditelne meni cenu INSERTu pri jednom indexe.

Meranie:

Experiment	Index	Pocet roznych hodnot	Total runtime
09	type	4	56.286 ms
10	supplier	138789	242.208 ms

V tomto merani bol INSERT s indexom nad `supplier` pomalsi ako INSERT s indexom nad `type`. Prakticky zaver je, ze index nad jednoduchym stlpcom s malym pocetom hodnot mal nizsiu cenu pri vkladani ako index nad stlpcom s vela roznyimi textovymi hodnotami.

Zaver

Indexy vyrazne pomohli pri selektivnych SELECT dotazoch. Pri `supplier = 'SPP'` sa plan zmenil zo sekvenčného skenu na bitmap index scan a dotaz bol radovo rychlejší. Pri range dotaze nad `total_amount` index tiež pomohol, ale menej, lebo dotaz vracal viac riadkov.

Na druhej strane INSERT benchmark ukázal opačný efekt: čím viac indexov bolo na tabulke, tým viac práce musel PostgreSQL urobiť pri vkladani riadkov. Indexy teda zrychlujú citanie, ale spomaľujú zápisy.